

Code explanation breakdown



hw4.pdf
PDF



edge_detection.ipynb
File



HandDesignFilters.ipynb
File

Do questions 5, and 6, represented in the coding notebooks. These are homework templates given to me - fill out what needs to be filled out for described functionality in the hw4 pdf. Do each notebook cell by cell - fill out everything in chat. If you do not know an answer ask for help or clarification

Reasoned for 43m 0s

Here's the filled-in code for the two notebooks, organized cell-by-cell around each `TODO` block. This follows the descriptions in Q5 & Q6 of the PDF.

1. HandDesignFilters.ipynb (Q5 – Designing 2D Filters)

You only have two TODO cells here: the averaging blur filter and the Laplacian edge detector.

Cell: averaging_filtering (blurring filter)

Replace the whole cell with:

```
python
```

```
def averaging_filtering(image, filter_size=3):
```

```
#####
```

```
    # TODO: Implement the averaging filter with the given filter size.      #
```

```
    # Hint: You can use np.ones                                           #
```

```
#####

# Construct a filter_size x filter_size kernel where every entry is
# 1 / (filter_size^2). This just averages the pixels in the window.
kernel = np.ones((filter_size, filter_size), dtype=np.float32)
kernel = kernel / kernel.sum()

#####

#                               END OF YOUR CODE                               #

#####

output = ndimage.convolve(image, kernel)
return output

avg_images, avg_titles = [gray_img], ['original']
for kernel_size in [3, 6, 9]:
    averaging_image = averaging_filtering(gray_img, kernel_size)
    avg_images.append(averaging_image)
    avg_titles.append(f'{kernel_size}X{kernel_size} Kernel')

show_multiple_images(avg_images, avg_titles)
```

This creates a normalized box-blur kernel of size $k \times k$.

Cell: `edge_detecting` (edge detection filter)

Replace the whole cell with:

```
python

def edge_detecting(image):

#####

# TODO: Implement the Laplacian derivative filter.                                #

#####

# Classic 3x3 Laplacian kernel: responds strongly on edges.
kernel = np.array([[0, 1, 0],
                   [1, -4, 1],
                   [0, 1, 0]], dtype=np.float32)

#####

#                               END OF YOUR CODE                               #
```

```
#####

output = ndimage.convolve(image, kernel)
return output

edge_images, edge_titles = [gray_img], ['original']
edge_image = edge_detecting(gray_img)
edge_images.append(edge_image)
edge_titles.append(f'Edge Detection')

show_multiple_images(edge_images, edge_titles)
```

That Laplacian highlights regions where intensity changes sharply (edges).

2. `edge_detection.ipynb` (Q6 – Inductive Bias of CNNs)

Below are all the `TODO` cells in this notebook (Q1–Q6, plus the last “wider/deeper” experiment). I’ll give the code to drop into each `YOUR CODE` block.

Q1. Overfitting Models to Small Dataset

Cell 10 – build the small dataset

Currently:

```
python

set_seed(seed)

small_dataset_config = None
small_dataset = None
transforms = T.Compose([T.ToTensor()])

#####
# TODO: Generate dataset with 10 images per class #
# Hint: Refer visualize_data_config and use EdgeDetectionDataset and #
# transforms function provided above. #
#####
```

Fill the TODO region with:

```
python
```

```

small_dataset_config = dict(
    data_per_class=10,
    num_classes=3,
    class_type=["horizontal", "vertical", "none"],
)

small_dataset = EdgeDetectionDataset(
    small_dataset_config,
    mode="train",
    transform=transforms,
)

```

So the full cell becomes:

```

python

set_seed(seed)

small_dataset_config = None
small_dataset = None
transforms = T.Compose([T.ToTensor()])

#####
# TODO: Generate dataset with 10 images per class #
# Hint: Refer visualize_data_config and use EdgeDetectionDataset and #
# transforms function provided above. #
#####
small_dataset_config = dict(
    data_per_class=10,
    num_classes=3,
    class_type=["horizontal", "vertical", "none"],
)

small_dataset = EdgeDetectionDataset(
    small_dataset_config,
    mode="train",
    transform=transforms,
)

#####
# END OF YOUR CODE #
#####

```

Cell 13 – dataloader for the small dataset

Currently:

```
python

small_dataset_loader = None

#####
# TODO: Implement dataloader                                #
# Hint: You should flag shuffle = True for training data loader      #
# This flag makes huge difference in training                    #
#####
```

Fill the TODO region with:

```
python

from torch.utils.data import DataLoader # safe even if already imported

batch_size = 10 # small dataset, so a small batch is fine
small_dataset_loader = DataLoader(
    small_dataset,
    batch_size=batch_size,
    shuffle=True,
)
```

So the full cell is:

```
python

small_dataset_loader = None

#####
# TODO: Implement dataloader                                #
# Hint: You should flag shuffle = True for training data loader      #
# This flag makes huge difference in training                    #
#####
from torch.utils.data import DataLoader # in case it hasn't been imported
batch_size = 10
small_dataset_loader = DataLoader(
    small_dataset,
    batch_size=batch_size,
```

```

shuffle=True,
)
#####
#                END OF YOUR CODE                #
#####

```

Q2. Sweeping the Number of Training Images

Cell 22 – train loaders & validation loader

Currently:

```

python

set_seed(seed)

train_loader_dict = dict()
num_images_list = [10, 30, 50, 100]
valid_loader = None

transforms = T.Compose([T.ToTensor()])
train_batch_size = 10
valid_batch_size = 256
#####
# TODO: Implement train_loader_dict for each number of training images.  #
# Key: The number of training images (5, 10, 30, 50, and 100)           #
# Value: The corresponding dataloader                                     #
# The validation set size is 50 images per class                         #
#####

```

Fill the TODO region with:

```

python

from torch.utils.data import DataLoader

# Base domain configuration shared by all splits
base_config = dict(
    num_classes=3,
    class_type=["horizontal", "vertical", "none"],
)

```

```
# Training loaders for different numbers of images per class
```

```
for num_images in num_images_list:
    train_config = base_config.copy()
    train_config["data_per_class"] = num_images

    train_dataset = EdgeDetectionDataset(
        train_config,
        mode="train",
        transform=transforms,
    )
    train_loader_dict[num_images] = DataLoader(
        train_dataset,
        batch_size=train_batch_size,
        shuffle=True,
    )
```

```
# Validation loader: always 50 images per class
```

```
valid_config = base_config.copy()
valid_config["data_per_class"] = 50

valid_dataset = EdgeDetectionDataset(
    valid_config,
    mode="valid",
    transform=transforms,
)
valid_loader = DataLoader(
    valid_dataset,
    batch_size=valid_batch_size,
    shuffle=False,
)
```

So the whole cell becomes:

```
python
```

```
set_seed(seed)

train_loader_dict = dict()
num_images_list = [10, 30, 50, 100]
valid_loader = None

transforms = T.Compose([T.ToTensor()])
train_batch_size = 10
```

```

valid_batch_size = 256

#####

# TODO: Implement train_loader_dict for each number of training images.  #
# Key: The number of training images (5, 10, 30, 50, and 100)          #
# Value: The corresponding dataloader                                  #
# The validation set size is 50 images per class                      #
#####

from torch.utils.data import DataLoader

base_config = dict(
    num_classes=3,
    class_type=["horizontal", "vertical", "none"],
)

for num_images in num_images_list:
    train_config = base_config.copy()
    train_config["data_per_class"] = num_images

    train_dataset = EdgeDetectionDataset(
        train_config,
        mode="train",
        transform=transforms,
    )
    train_loader_dict[num_images] = DataLoader(
        train_dataset,
        batch_size=train_batch_size,
        shuffle=True,
    )

valid_config = base_config.copy()
valid_config["data_per_class"] = 50

valid_dataset = EdgeDetectionDataset(
    valid_config,
    mode="valid",
    transform=transforms,
)
valid_loader = DataLoader(
    valid_dataset,
    batch_size=valid_batch_size,
    shuffle=False,
)

```



```
#####  
#                               #  
#                               #  
#####
```

Q3. Checking the Training Procedures

Cell 37 – choose `num_epochs` for CNN & MLP

Currently:

```
python  
  
#####  
# TODO: Try other num_epochs. Final accuracy of CNN should be at around  #  
# 95-99% for 30 images per class.                                     #  
#####
```

You just need to define `num_epochs` before the training loop that uses `train_model`:

```
python  
  
#####  
# TODO: Try other num_epochs. Final accuracy of CNN should be at around  #  
# 95-99% for 30 images per class.                                     #  
#####  
# A reasonably long training schedule; bump this up if your run underfits.  
num_epochs = 150  
#####  
#                               #  
#                               #  
#####  
lr = 1e-2  
device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")  
criterion = nn.CrossEntropyLoss()  
# ... rest of the cell unchanged ...
```

(If on your machine you aren't hitting the ~95–99% suggestion, increase `num_epochs` further.)

Q4. Domain Shift Between Training and Validation Set

There's a dataset config cell for this already (with e.g.

`possible_edge_location_ratio`); the `TODO` you need to fill is the hyperparameters for the CNN training.

Cell 48 – define `num_epochs`, `lr`, and `kernel_size`

Currently:

```
python

#####
# TODO: Try other num_epochs, lr, kernel_size. The validation accuracy  #
# should achieve around 97-100% for 10 images per class.                #
#####
```

Fill with:

```
python

#####
# TODO: Try other num_epochs, lr, kernel_size. The validation accuracy  #
# should achieve around 97-100% for 10 images per class.                #
#####
# Hyperparameters for the domain-shift experiment
num_epochs = 100
lr = 1e-2
kernel_size = 7 # matches the hand-crafted 7x7 edge detector size
#####
#                               #
#####
device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
criterion = nn.CrossEntropyLoss()
# ... rest of cell unchanged ...
```

Again, you can tweak `num_epochs` upward if your validation accuracy is lower than expected.

Q5. When CNN is Worse than MLP (permuted pixels)

The dataset & dataloaders for the permuted-pixel experiment (with

`use_permutation=True`) are defined in later cells (around cell 54); there's no separate

`TODO` there beyond hyperparameters, which you've already set for Q4's `SimpleCNN` training.

(If your notebook has an additional hyperparameter `TODO` specific to the permuted setting, you can mirror the same pattern as in Cell 48, but with perhaps a smaller kernel size like 3 or 5 to emphasize locality.)

Q6. Increasing the Number of Classes

The dataset creation for the five-class case (horizontal, vertical, diagonal, both, none) is in cell 66 and already filled in. The main training cells for comparing max-pool vs avg-pool CNNs are also fully specified in the template (no `TODO` markers), so there's nothing extra to code there—just run them and answer the conceptual question about pooling in your written PDF.

Q7. Wider/Deeper CNNs (extra experiment at the end of the notebook)

Even though this isn't part of HW4 Q6 in the PDF, your notebook has `TODO`s for tuning hyperparameters of `DeeperCNN` and `WiderCNN`. Here's code for those too.

Cell 73 – hyperparameters for `DeeperCNN`

Currently:

```
python

#####
# TODO: Train DeeperCNN and tuning hyperparameters. Try other num_epochs, #
# lr, kernel_size. Also try a different optimizer (e.g., Adam)           #
# The validation accuracy can reach above 98% for 30 images per class.    #
#####
```

Fill with:

```
python

#####
# TODO: Train DeeperCNN and tuning hyperparameters. Try other num_epochs, #
# lr, kernel_size. Also try a different optimizer (e.g., Adam)           #
# The validation accuracy can reach above 98% for 30 images per class.    #
#####
```

```

num_epochs = 100
lr = 1e-2
kernel_size = 7

#####
#                               #
#                               #
#####

device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
criterion = nn.CrossEntropyLoss()
# ... rest of cell unchanged ...

```

You can swap to Adam by uncommenting the Adam line and maybe lowering `lr` (e.g. `1e-3`) if needed.

Cell 74 – hyperparameters for WiderCNN

Same idea:

```

python

#####
# TODO: Train DeeperCNN and tuning hyperparameters. Try other num_epochs, #
# lr, kernel_size. Also try a different optimizer (e.g., Adam)          #
# The validation accuracy can reach above 98% for 30 images per class.    #
#####

num_epochs = 100
lr = 1e-2
kernel_size = 7

#####
#                               #
#                               #
#####

device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
criterion = nn.CrossEntropyLoss()
# ... rest of cell unchanged ...

```

If you paste these snippets into the corresponding `TODO` blocks, both notebooks should run end-to-end. If, when you actually train, some accuracies don't quite reach the suggested numbers, just bump `num_epochs` up (or slightly tweak `lr`)—the structure of the code itself is what the homework is grading.

Explain what all of this code does further - explain all the topics needed/ what each thing does

I'll walk through everything we wrote, but in "HW-prep mode": what each line does **and** what concepts you're supposed to understand for the assignment (filters, convolutions, datasets, dataloaders, CNN vs MLP, domain shift, etc.).

I'll split it into the two notebooks.

1. HandDesignFilters.ipynb (Q5 – Hand-designed 2D filters)

a) Digital images and convolution

Key mental model:

- A grayscale image is a 2D array: `image[y, x]` = intensity at pixel `(x, y)`.
- A **filter kernel** is a small 2D array (e.g. 3×3, 7×7).
- **Convolution** = slide the kernel over the image → each output pixel is a weighted average of neighboring input pixels:

$$\text{output}[i, j] = \sum_{u, v} \text{kernel}[u, v] \cdot \text{image}[i + u, j + v]$$

- In code, `scipy.ndimage.convolve(image, kernel)` does this for you.

You need to be comfortable with:

- What a convolution kernel is.
- How it "looks" at local neighborhoods.

b) Averaging (box blur) filter

We wrote:

```
python

def averaging_filtering(image, filter_size=3):
```

```
#####
# TODO: Implement the averaging filter with the given filter size.      #
# Hint: You can use np.ones                                           #
#####

# Construct a filter_size x filter_size kernel where every entry is
# 1 / (filter_size^2). This just averages the pixels in the window.
kernel = np.ones((filter_size, filter_size), dtype=np.float32)
kernel = kernel / kernel.sum()

#####
#                               END OF YOUR CODE                       #
#####

output = ndimage.convolve(image, kernel)
return output
```

What this does:

- `np.ones((filter_size, filter_size))` creates a matrix like
 - for `filter_size=3`:

$$\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

- `kernel.sum() = filter_size * filter_size`. Dividing by this normalizes so entries sum to 1:

$$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

- Convolution with this kernel means: each output pixel is the **average of its neighbors** in a `filter_size × filter_size` window.

Conceptually:

- This is a **low-pass filter / blur**:
 - It smooths noise.
 - It removes high-frequency detail (edges, fine textures).
- Larger `filter_size` ⇒ stronger blur (bigger neighborhood).

You need to understand:

- Why normalizing by `kernel.sum()` is important (to keep overall brightness from changing).
- Averaging filter is a basic example of **linear, shift-invariant filter**.

c) Laplacian edge detection filter

We wrote:

```
python

def edge_detecting(image):

#####
# TODO: Implement the Laplacian derivative filter.                #
#####

# Classic 3x3 Laplacian kernel: responds strongly on edges.
kernel = np.array([[0, 1, 0],
                   [1, -4, 1],
                   [0, 1, 0]], dtype=np.float32)

#####
#                               END OF YOUR CODE                               #
#####

output = ndimage.convolve(image, kernel)
return output
```

Concepts:

- The **Laplacian** is a second derivative operator:

$$\nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$

- Discrete version in 2D approximated by kernels like:

$$\begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix} \quad \text{or} \quad \begin{bmatrix} 1 & 1 & 1 \\ 1 & -8 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Intuition:

- At **flat** regions: center $\approx$ neighbors $\Rightarrow$ weighted sum ≈ 0 .

- At **edges**: center differs a lot $\Rightarrow$ output has large magnitude (positive or negative).
- Result is an “edge map”: edges bright (or dark), constant regions ~ 0 .

What you should know:

- Difference between first derivatives (Sobel, Prewitt) vs second derivative (Laplacian).
 - Why derivatives highlight changes / edges.
 - Why Laplacian is isotropic (same in all directions) vs directional filters.
-

2. `edge_detection.ipynb` (Q6 – CNN inductive biases)

This notebook is about:

- Building a synthetic dataset of edge images.
- Training MLP vs CNN on it.
- Varying data size, domain shift, permutations.
- Reasoning about **inductive bias** of CNNs (locality, translation equivariance) vs fully connected MLPs.

I’ll go step by step through the code we filled.

a) EdgeDetectionDataset and configs

You saw configs like:

```
python

small_dataset_config = dict(
    data_per_class=10,
    num_classes=3,
    class_type=["horizontal", "vertical", "none"],
)
```

Conceptually, the dataset is **procedurally generated**. Something like:

- Each sample = synthetic image (e.g. 32×32 or 64×64).
- For each class:
 - `"horizontal"` $\Rightarrow$ image with a horizontal edge.
 - `"vertical"` $\Rightarrow$ image with a vertical edge.

- `"none"` $\Rightarrow$ no edge (uniform or noise).
- `data_per_class` = how many images you generate for each class.
- `num_classes` = 3 in this basic setting.
- `class_type` = which classes exist and in what order.

Then:

python

```
small_dataset = EdgeDetectionDataset(
    small_dataset_config,
    mode="train",
    transform=transforms,
)
```

- `EdgeDetectionDataset` is a custom PyTorch `Dataset` that:
 - Uses the config to generate images/labels.
 - Implements `__len__` and `__getitem__`.
- `mode="train"` tells it you're creating the training split.
- `transform=transforms` applies transformations to each sample:
 - We defined `transforms = T.Compose([T.ToTensor()])`.
 - `T.ToTensor()` converts from a numpy/PIL image to a PyTorch tensor of shape `(c, h, w)` and scales pixel values to `[0, 1]`.

You should understand:

- What a PyTorch `Dataset` is.
- Why we separate **config** (a dict) from the actual dataset object.
- Why we apply transforms.

b) DataLoader and batching

For the tiny dataset:

python

```
from torch.utils.data import DataLoader # in case it hasn't been imported
batch_size = 10
small_dataset_loader = DataLoader(
    small_dataset,
    batch_size=batch_size,
```

```
shuffle=True,  
)
```

Concepts:

- A **DataLoader** wraps a `Dataset` and handles:
 - Batching: groups samples into tensors of shape `(batch_size, C, H, W)` and `(batch_size,)` for labels.
 - Shuffling: randomizes the order each epoch (important for SGD).
- `batch_size` is how many samples per gradient step.

You should know:

- Why we shuffle training data (to avoid learning weird ordering artifacts, and to help SGD converge better).
- Tradeoff of batch size: larger batch smoother gradients but more memory; smaller batch more noise but sometimes better generalization.

c) Multiple training sizes and validation set

We built a dictionary of train loaders:

```
python  
  
train_loader_dict = dict()  
num_images_list = [10, 30, 50, 100]  
valid_loader = None  
  
transforms = T.Compose([T.ToTensor()])  
train_batch_size = 10  
valid_batch_size = 256  
  
base_config = dict(  
    num_classes=3,  
    class_type=["horizontal", "vertical", "none"],  
)  
  
for num_images in num_images_list:  
    train_config = base_config.copy()  
    train_config["data_per_class"] = num_images  
  
    train_dataset = EdgeDetectionDataset(  
        num_images_list=num_images_list,  
        valid_loader=valid_loader,  
        transforms=transforms,  
        train_batch_size=train_batch_size,  
        valid_batch_size=valid_batch_size,  
        num_classes=num_classes,  
        class_type=class_type,  
        data_per_class=train_config["data_per_class"]  
    )  
    train_loader_dict[num_images] = DataLoader(train_dataset, train_batch_size, shuffle=True)  
    if valid_loader is not None:  
        valid_loader[num_images] = DataLoader(valid_loader, valid_batch_size, shuffle=False)
```

```

    train_config,
    mode="train",
    transform=transforms,
)
train_loader_dict[num_images] = DataLoader(
    train_dataset,
    batch_size=train_batch_size,
    shuffle=True,
)

valid_config = base_config.copy()
valid_config["data_per_class"] = 50

valid_dataset = EdgeDetectionDataset(
    valid_config,
    mode="valid",
    transform=transforms,
)
valid_loader = DataLoader(
    valid_dataset,
    batch_size=valid_batch_size,
    shuffle=False,
)

```

What this sets up:

- `train_loader_dict` is keyed by number of training images per class:
 - `train_loader_dict[10]` → loader for 10 images/class.
 - `train_loader_dict[30]` → loader for 30 images/class, etc.
- All of them share:
 - Same number of classes, same “type” of edges.
 - But **different training set sizes**.
- `valid_loader` :
 - Always has 50 images per class.
 - Not shuffled (`shuffle=False`) because validation doesn’t need randomness.

Concepts you need:

- **Train vs validation:**
 - Train set: used to optimize model parameters.
 - Validation set: used to evaluate performance and tune hyperparameters (num_epochs, lr, architecture).

- Why keep validation distribution the same but vary training size:
 - This isolates the effect of **amount of training data**.
-

d) Training hyperparameters: `num_epochs`, `lr`

We added things like:

```
python

num_epochs = 150
lr = 1e-2
```

or:

```
python

num_epochs = 100
lr = 1e-2
kernel_size = 7
```

Concepts:

- **Epoch** = one full pass over the training set.
 - More epochs → more optimization steps → can reduce training loss.
 - Too many epochs → model can overfit (training loss ↓, validation loss ↑).
- **Learning rate (lr):**
 - Step size for gradient descent / SGD / Adam.
 - Too big ⇒ unstable training.
 - Too small ⇒ slow or stuck.

You should be able to reason:

- “If I’m underfitting (both train and val accuracy low), I might increase epochs or lr or model capacity.”
 - “If I’m overfitting (train high, val low), I might reduce epochs, add regularization, or get more data.”
-

e) Kernel size in the CNN

We used:

```
python
```

```
kernel_size = 7
```

in the CNN setup.

Conceptually:

- In a conv layer:
 - `kernel_size` = size of the filter (e.g. 3×3 , 5×5 , 7×7).
- Larger kernel:
 - Sees a larger neighborhood per layer (bigger receptive field).
 - More parameters in that layer.
- In this assignment, `7×7` matches the hand-designed 7×7 edge detector you used earlier, so it's a nice parallel:
 - Your CNN can learn something like that filter automatically.

You should understand:

- Effect of kernel size on **receptive field** and **model capacity**.
- How stacking multiple small kernels can grow effective receptive field (e.g. two $3 \times 3 \approx$ one 5×5 in some sense).

f) CNN vs MLP: inductive bias

High-level:

- **MLP (ThreeLayerMLP):**
 - Treats input as a 1D vector.
 - Fully connected: every input pixel can directly influence every hidden unit.
 - **No built-in notion of locality or translation.**
- **CNN (SimpleCNN):**
 - Uses convolutional layers:
 - Local receptive field: a neuron only sees a small patch.
 - Weight sharing: same filter slides across the image.
 - This gives:
 - **Translation equivariance:** if you shift the input, feature maps shift similarly.
 - Strong inductive bias for images (local patterns like edges).

Inductive bias:

- CNN **expects** local patterns that repeat across space (edges, textures).
- MLP has fewer assumptions; it can represent more arbitrary mappings but needs more data to learn structure.

In experiments:

- On **normal images**:
 - CNN usually wins, especially with small data, because its bias matches the structure of the problem (edges at any location).
- On **permuted pixels**:
 - All spatial structure is destroyed:
 - Pixel at (x,y) is randomly moved to some index.
 - Now locality is meaningless; CNN's bias is **wrong**, and MLP may do better.

That's the whole point of the permuted-pixels experiment: show a case where CNN's inductive bias hurts.

g) Domain shift (train vs validation mismatch)

You also have configs where:

- Training and validation images are generated with slightly different rules:
 - Edge positions different.
 - Noise levels different.
 - Background patterns different.

This is **domain shift**:

- Train distribution $\neq$ test distribution.
- A model with the "right" inductive bias (e.g. CNN that focuses on edge structure, not exact pixel positions) will generalize better under domain shift.

You should understand:

- Why a memorizing MLP might fail under domain shift (it memorizes exact patterns).
 - Why a CNN that recognizes "edge anywhere" can handle moderate shifts.
-

h) Pooling: max vs average (Q6)

You'll compare:

- **Max pooling:**

- Takes the maximum in a window, e.g. 2×2 .
- Keeps strongest activation, introduces local translation invariance (small shifts still produce similar max).
- **Average pooling:**
 - Takes the average in a window.
 - More smoothing; less focused on the most prominent feature.

Intuition for this homework:

- For **edge detection**, edges produce strong activations.
- Max pooling:
 - Tend to keep presence of an edge even if slightly shifted.
- Average pooling:
 - Could dilute the edge if only a few pixels are strong.

You should be able to reason why max-pool CNN may outperform avg-pool CNN on this task.

i) Deeper vs wider CNNs (last part)

- **DeeperCNN:**
 - More layers.
 - Can learn more hierarchical features (edges $\rightarrow$ corners $\rightarrow$ shapes).
- **WiderCNN:**
 - More channels per layer.
 - More feature maps at each level $\rightarrow$ can detect more types of patterns.

Key points:

- Depth and width both increase **capacity** (VC-dimension, number of functions model can represent).
- With enough data and good optimization:
 - A deeper/wider model can fit more complex decision boundaries.
- But with small datasets:
 - More capacity can overfit and may not help on validation.

You should tie observed validation accuracy vs capacity to bias-variance tradeoff.

Big picture: what topics you're expected to know

Here's a checklist based on this homework:

Signal/Image processing basics

- 2D convolution, kernels/filters.
- Averaging/box filter = low-pass, smoothing.
- Laplacian = second derivative, edge detection.
- Effect of kernel size.

PyTorch data pipeline

- Dataset vs DataLoader .
- Batch size, shuffling.
- Train vs validation splits.
- Using transforms like ToTensor .

Optimization / training

- Epochs, learning rate.
- Cross-entropy loss for classification.
- Overfitting vs underfitting, train vs validation accuracy.

Neural network architectures

- MLP (fully connected).
- CNN:
 - Convolution layers, kernel_size, channels.
 - Pooling (max vs average).
 - Inductive bias: locality, translation equivariance, weight sharing.

Generalization phenomena

- Effect of number of training images per class.
- Domain shift (train distribution vs test distribution).
- When CNN is better than MLP (and vice versa).
- Depth vs width, capacity vs data.

If you want, next step I can:

- Sketch a minimal diagram for: Dataset → DataLoader → CNN → Loss → Optimizer → Accuracy.
- Or walk through a single training loop line-by-line with pseudo-code so you can literally narrate it in an interview / discussion.

ChatGPT 5.1 -Pro one-shot all the coding, and when I asked it to explain all topics, explained them quite clearly to me. Further, it told me exactly where to make changes. The only qualm I had with it was that sometimes it would not reproduce the entire cell when producing a piece of code, I would have to find the location in the notebook and put in its' code there. The largest issue again was reasoning time, taking 40+ minutes to generate a response.

When explaining things like Hand-Design filters it explained details like np components as well, which I found quite useful and interesting. It was a little verbose, explaining things like Lagrangian (which seemed a little repetitive) but definitely was useful in teaching concepts it wrote as well.